```
        """
        Upgrade existing DBs from lat_e5/lon_e5 to lat_e6/lon_e6 without data loss.
        """
        # flight_latest
        cols = self._table_columns(cur, "flight_latest")
        if "lat_e6" not in cols:
            cur.execute("ALTER TABLE flight_latest ADD COLUMN lat_e6 INTEGER")
        if "lon_e6" not in cols:
            cur.execute("ALTER TABLE flight_latest ADD COLUMN lon_e6 INTEGER")

        cols = self._table_columns(cur, "flight_latest")
        if "lat_e5" in cols and "lon_e5" in cols:
            cur.execute(
                """
                UPDATE flight_latest
                SET
                    lat_e6 = COALESCE(lat_e6, lat_e5 * 10),
                    lon_e6 = COALESCE(lon_e6, lon_e5 * 10)
                """
            )

        # trajectory_samples
        cols = self._table_columns(cur, "trajectory_samples")
        if "lat_e6" not in cols:
            cur.execute("ALTER TABLE trajectory_samples ADD COLUMN lat_e6 INTEGER")
        if "lon_e6" not in cols:
            cur.execute("ALTER TABLE trajectory_samples ADD COLUMN lon_e6 INTEGER")

        cols = self._table_columns(cur, "trajectory_samples")
        if "lat_e5" in cols and "lon_e5" in cols:
            cur.execute(
                """
                UPDATE trajectory_samples
                SET
                    lat_e6 = COALESCE(lat_e6, lat_e5 * 10),
                    lon_e6 = COALESCE(lon_e6, lon_e5 * 10)
                """
            )

    def _refresh_schema_flags(self, cur):
        flight_cols = self._table_columns(cur, "flight_latest")
        traj_cols = self._table_columns(cur, "trajectory_samples")
        self._flight_has_lat_e5 = "lat_e5" in flight_cols
        self._flight_has_lon_e5 = "lon_e5" in flight_cols
        self._traj_has_lat_e5 = "lat_e5" in traj_cols
        self._traj_has_lon_e5 = "lon_e5" in traj_cols

    def _quantize(self, row):
        lat = _to_float(row.get("latDD"), 0.0)
        lon = _to_float(row.get("lonDD"), 0.0)
        alt_m = int(round(_to_float(row.get("altitudeMM"), 0.0) / 1000.0))
        return {
            "lat_e6": int(round(lat * 1000000)),
            "lon_e6": int(round(lon * 1000000)),
            "altitude_m": alt_m,
            "speed_cms": _to_int(row.get("horVelocityCMS"), 0),
            "heading_de2": _to_int(row.get("headingDE2"), 0),
            "vs_cms": _to_int(row.get("verVelocityCMS"), 0),
            "squawk": _to_int(row.get("squawk"), 0),
        }

    def _should_store(self, flight_key, ts_iso, q):
        prev = self._last_saved.get(flight_key)
        if prev is None:
            return True

        prev_dt = prev["dt"]
        cur_iso, cur_dt = _parse_ts(ts_iso)
        if cur_dt is None or prev_dt is None:
            return True

        elapsed = (cur_dt - prev_dt).total_seconds()
        if elapsed >= self.min_interval_sec:
```